

Playwright-Notes-2

- **Installation- (In Playwright-Practice folder in VSCode)**

1. Install **Playwright Test for VSCode** extension in VSCode
2. In Git bash terminal in VSCode run the following command to install Playwright- **npm init playwright@latest**
3. The following get installed:
playwright.config.ts # Test configuration
package.json
package-lock.json # Or yarn.lock / pnpm-lock.yaml
tests/
 example.spec.ts # Minimal example test

Note: To comment and uncomment code, press 'ctrl+/'

When prompted, choose / confirm:

- TypeScript or JavaScript (default: TypeScript)
- Tests folder name (default: `tests`, or `e2e` if `tests` already exists)
- Add a GitHub Actions workflow (recommended for CI)
- Install Playwright browsers (default: yes)

Note: npx-(node package executor), npm-(node package manager)

- **Creating and running a test file-**

1. Test files always end with the extension **.spec.ts**
2. Create a test file in the /test folder and name it with the extension **.spec.ts**
3. Import {test} component from the Playwright folder-**import {test} from '@playwright/test';**
4. Write the test script/test case-in the test block/test function, provide the name of the test ('**My Test**') and the anonymous function (a function without a name-() $\Rightarrow$ {}) as the parameters-(**2 parameters required in the test block**)

5. Write the test scripts inside the test block-(test scripts can be written to perform any automated activity/action like opening an application, clicking on a button, providing user credentials like username, password,etc and pressing the login button to log into the application)
6. Similarly provide multiple test scripts/test cases to perform various activities/actions
7. Running the command **npx playwright test** will run all the test files (.spec.ts) in the test folder
8. Running the command **npx playwright test filename.spec.ts** will run a specific test file
9. Running the command **npx playwright show-report** shows the HTML test report of your tests after running them to understand which test passed and which test failed and the time taken for execution by each test

- **async/await in JS and TS and page fixture in playwright-**

1. Mark the anonymous function which you provided as a parameter to the test script as an async function and provide a page fixture to it (pass {page} as a parameter to the async function like so (test block containing the different test scripts/test cases inside it):

```
test(<name of your test>, async({page})=>{
```

```
// test scripts/test cases
```

```
})
```

```
Eg: test(<'My Test'>, async({page})=>{
```

```
// test scripts/test cases
```

```
})
```

2. Use the page fixture to open the browser and navigate to the provided url (www.google.com) using `.goto()` method-**`await page.goto("https://www.google.com")`**

3. Run the test script which you have just created and enable the **Show browser** setting under **Playwright settings**

4. Write a test script to open Google browser and click on the **Google Apps icon**. Obtain its location using the **Pick Locator** setting under **Playwright settings**. From the **Playwright terminal** (at the bottom near terminal in VSCode) copy the location of the Google Apps icon-it should be-**`getByRole('button', { name: 'Google apps' })`**

5. Using the page fixture, write a script to open Google browser and click on the Google Apps Icon-**`await page.<location of GoogleApps button>.click()=>await page.getByRole('button', { name: 'Google apps' }).click()`**

6. The `.goto()` and `.click()` methods return promises

7. **`await()`** in this case waits until the promise has been completed before executing the next line. If we don't use `await()` all the lines get executed together simultaneously and we can't view the automated tasks/actions that Playwright is doing as the browser automatically closes simultaneously.

So in order to view each automated task/action that Playwright does, we use `await()` as it executes each line one by one.

8. `{page}` fixture- A pre-made browser tab that's automatically given to your test, and cleaned up when the test is done. You don't open the browser, you don't create the tab, you don't close it — Playwright does all that for you. You just use `page` to interact with it. Think of it like a hotel room — you didn't build it, you don't clean it up, you just walk in and use it. When you check out (test ends), it's handled for you.

Each test gets its own fresh tab, so tests don't interfere with each other. `{page}` is an argument which tells Playwright to set up the page fixture and provide it to your

test function/test block. Playwright will setup the page fixture before running the test and tear it down after the test has finished. Test fixtures are isolated between tests.

9. If we don't use {page} fixture we can alternatively write the below 3 lines of code to create a browser and perform actions without using {page} fixture:

```
const browser=await chromium.launch();  
const context=await browser.newContext();  
const page=await context.newPage();
```

It essentially does the same thing-**opening the Google browser and clicking on the Google Apps icon**

- **Locators**-represent a way to find elements on a page at any moment, like buttons, search bar, etc. Locators can be created with the **page.locator()** method
- **Locator method**-returns an element locator that can be used to perform actions on a page/frame
- **Locator method usage:** **page.locator(selector)**, **page.locator(selector, options)**

- **Types of locators:**

1. XPath-navigates through a page's HTML structure

Syntax:

```
//htmltag["@attribute='attributeValue']
```

(or)

```
//*[@attribute=attributevalue"]
```

-to understand how it works, right click>select inspect>locate any specific element and press ctrl+f> then you can search the element by the XPath using the above syntax>it then highlights the html element

In VSCode:

1. Create a new test file (.spec.ts) in the tests folder
2. Import {test} component from the playwright/test folder/directory like so:

```
import {test} from '@playwright/test'
```

3. Create a test block with the name of the test and the async function that takes {page} fixture as its parameter
4. Inside the test block write a test script to navigate to the website **https://www.saucedemo.com/** using the .goto method:

```
await page.goto("https://www.saucedemo.com/")
```

5. Write a test script to locate the element using the **.locator()** method. Inside the .locator() method, provide the location of the element using the XPath syntax. In this case we want to locate the placeholder for entering the username (filling the username placeholder with a value -for filling the placeholder we use **.fill(<string_value>)** method which takes a string as a parameter (username that needs to be filled in the placeholder):

```
await  
page.locator("//*[@placeholder='Username']").fill("standa  
rd_user")
```

2. CSS Selectors-string patterns used to identify elements based on a combination of html tags, id, class, attributes

Syntax:

-htmltag.classValue (or) .classValue

-htmltag#idValue (or) #idValue

-htmltag[attributeName=attributeValue] (or)

[attributeName=attributeValue]

In VSCode:

Write a test script to locate the password placeholder/field and fill it with the password. Locate the element using the **.locator()** method. Inside the **.locator()** method, provide the location of the element using the CSS syntax. In this case we want to locate the placeholder for entering the password (filling the password placeholder with a value -for filling the placeholder we use **.fill(<string_value>)** method which takes a string as a parameter (password that needs to be filled in the placeholder) We use the **#idValue** CSS locator syntax for locating the element:

await page.locator("#password").fill("secret_sauce")

Note: whenever you see spaces inside class=" ", each word is a separate class name.

Write a test script to locate the **Login** button and click on it using **.click()** method. Locate the element using the **.locator()** method. Inside the **.locator()** method, provide the location of the element using the CSS syntax. We use the **.ClassValue** CSS locator syntax for locating the element:

await page.locator(".submit-button").click()

(or)

await page.locator("[value=Login]").click()=>CSS locator using **[attributeName=attributeValue]** syntax

3. Text selectors-used to select text elements like buttons containing text which are mainly links-used to locate/identify text elements

Syntax:

text='textValue' (case sensitive)

(or)

text=textValue (not case sensitive)

In VSCode:

Write a test script to locate the Sauce Labs Backpack link and click on it using **.click()** method. Locate the element using the **.locator()** method. Inside the **.locator()** method, provide the location of the element using the text syntax. We use the **text='textValue'** text locator syntax for locating the element:

```
await page.locator("text='Sauce Labs Backpack']").click()
```

Note: Use text selectors only for clicking on links with text or buttons with text-it doesn't locate placeholders and fill them as placeholders are attributes and not explicit text so it can't locate placeholders. We use getBy methods instead for placeholders apart from XPath and CSS selectors

NOTE: It's not recommended to use XPath and CSS locators as DOM elements can change. Use getBy locator methods instead which we will learn further

NOTE: .=>for class, #=>for id, //=>for attribute

- **Locator method usage with page.locator(selector, options=>syntax)-options is an optional argument that can be given with the .locator() method-mainly used to locate the closest div element that contain a particular id, class or attribute of the element (this topic isn't that important)**
- **Types of option arguments that can be used with .locator() method:**

1. has-locates elements containing an element that matches an inner locator element

Eg:

```
await  
page.locator(".form_group",{has:page.locator("#user-na  
me"}}).click()
```

await

```
page.locator(".form_group",{has:page.locator("#user-name"))}.pressSequentially("standard_user")
```

Here we locate the closest **div** element (class, we can also use id or attribute as well) **.form_group** and match it with the inner locator which is an **id #user-name** using **has**, click on it (using **.click() method**) and we fill the placeholder with the value **"standard_user"** as the username using **pressSequentially()** method. (Here **.fill()** won't work as we are locating a **div** element in the outer **.locator()** method and not a placeholder and **.fill()** method works at a DOM level whereas **.pressSequentially()** works at the keyboard level, simulating actual keyboard events.

2. hasNot-locates elements containing an element that doesn't match an inner locator element

Eg:

await

```
page.locator(".form_group",{hasNot:page.locator("#user-name"))}.click()
```

await

```
page.locator(".form_group",{hasNot:page.locator("#user-name"))}.pressSequentially("secret_sauce")
```

Here, since there are two **div** elements containing the class **.form_group** (one for **username** and one for **password**) we want to detect the **.form_group** div element containing the password, click on it (using **.click() method**) and fill it with the password **"secret_sauce"** and we don't want to detect the **.form_group** div element containing username so that's

why we use the **hasNot** option to prevent the detection of the `.form_group` div element containing username.

3. hasText-locates elements containing an element that matches a inner specified text

Eg:

await

```
page.locator(".inventory_item_name",{hasText:"Sauce  
Labs Fleece Jacket"}).click()
```

Here we locate the closest div element containing the class `.inventory_item_name` and match it with the element that contains the text **"Sauce Labs Fleece Jacket"** (mostly a link) and we click on it using the `.click()` method

4. hasNotText-locates elements containing an element that doesn't match a inner specified text

Eg:

await

```
page.locator(".inventory_item_name",{hasNotText:"Sauc  
e"}).click()
```

Here we locate the closest div element containing the class `.inventory_item_name` and filter out the elements that contain the text **"Sauce"** (because our intention is to click on the link/text element that doesn't contain the word **"Sauce"** and that link is **"Test.allTheThings() T-Shirt (Red)"**) and we click on it using the `.click()` method

- **getBy methods for locating an element**

1. getByLabel-locates input elements using the **text of the associated <label>** or **aria-labelledby** element or by the **aria-label** attribute.

-Accessible Rich Internet Applications (ARIA) is a set of roles and attributes that define ways to make web content and web applications more accessible to people with disabilities.

-Use this locator when locating form fields

In VSCode:

First, we write a test script to navigate to the website **<https://demo.nopcommerce.com/login>** using the **.goto()** method like so:

```
await page.goto("https://demo.nopcommerce.com/login")
```

Next, we write a test script to locate the form field that is labelled as **"Email:"** using the **getByLabel()** method and we fill it with the email id value **ruxstudent@gmail.com** using the **.fill()** method like so:

```
await page.getByLabel("Email:").fill("ruxstudent@gmail.com")
```

(or)

```
await page.getByLabel("Email").fill("ruxstudent@gmail.com") => (this is also acceptable)
```

2. getByPlaceholder-locates input elements using the **placeholder text**

-Use this locator when locating form elements that don't have labels but have placeholder texts

In VSCode:

We write a test script to locate the placeholder element that contains the placeholder text **"Search Store"** using the **getByPlaceholder()** method and we fill it with the value/placeholder text **Walmart** using the **.fill()** method like so:

```
await page.getByPlaceholder("Search store").fill("Walmart")
```

3. getByText-locates elements that contain a specific text between div or span elements, links, text between the button tags, etc

-You can match an exact string, substring or a regular expression using this method.

-Use text locators to find non-interactive elements like div, span, p, etc. For interactive elements such as buttons, links, inputs use Role locator

In VSCode:

We write a test script to locate the text **"Welcome, Please Sign In!"** and print its content using the **textContent()** method:

Note: textContent is a JavaScript/Playwright property that retrieves the text written inside an HTML element.

```
console.log(await page.getByText("Welcome, Please Sign In!").textContent())
```

Eg: await page.getByText("Jewelry").click()=>clicks on the category link "Jewelry"

4. getByAltText-locates elements by their alt text (mainly used for images)

-Use this locator to find image or area elements using their alt text

In VSCode:

```
Eg: await page.getByAltText("nopCommerce demo store").click()
```

Here we write a test script to locate the alt text for the logo and click on it

5. getByTitle-locates elements by their **title** attribute

-Use this locator when the element that needs to be located has a title attribute

In VSCode:

Eg: `await page.getByTitle("Show products in category Apparel").first().click()`

Here we write a test script to locate the text "Show products in category Apparel" as it contains a **title** attribute and we click on it. We include **.first()** method so we can locate the first occurrence of this text "Show products in category Apparel" and click on it (as this text is present multiple times so to avoid confusion we click on its first occurrence using the **.first()** method)

6. `getByRole`-locates elements by their ARIA role, ARIA attributes and accessible name

-Role locators include buttons, checkboxes, headings, lists, links, tables and many more and follow ****W3C specifications** (see below for more information) for ARIA role, ARIA attributes and accessible name

-It is highly recommended to use Role locators most of the time while locate elements as it is the closest way to how users and assistive technologies perceive the page

In VSCode:

Eg: `await page.getByRole("button", {name:"Books"}).click()`

Here, we locate the link "Books" using **getByRole**, obtain its role (**this can be obtained via the HTML tag which the element is enclosed in or the attribute Role if the element has the Role attribute specified**) and we click on it using the **.click()** method

****W3C specifications:**The W3C (World Wide Web Consortium) specifications are formal technical documents that define the standards and guidelines for web technologies. They ensure the web works consistently across browsers, devices, and platforms.

What They Cover

Core web languages

HTML, CSS, XML, and SVG – the building blocks of web pages.

APIs and interactivity

DOM (Document Object Model), Web Components, Web Storage, WebSockets, and more.

Accessibility

WCAG (Web Content Accessibility Guidelines) – rules for making the web usable by people with disabilities.

Linked data & semantics

RDF, OWL, SPARQL – technologies for the "Semantic Web" and structured data.

Security & privacy

Content Security Policy (CSP), WebAuthn, permissions APIs.

Performance

Resource Timing, Navigation Timing, and related APIs.

Why They Matter

Browsers like Chrome, Firefox, and Safari implement W3C specs so web pages behave consistently everywhere.

Developers rely on them to write interoperable, future-proof code.

They are developed openly, with input from browser vendors, companies, accessibility advocates, and the public.

7. `getById`-locates an element based on its testID which is the **`data-test`** attribute that has been specified for the element

-Use this locator when you can't locate an element by role or text and if the element has the attribute data-test specified

In VSCode:

```
await page.goto("https://www.saucedemo.com/")
```

```
await page.getByTestId("username").pressSequentially("rux0422")
```

First add this to the playwright.config.ts file under use:{} =>
testIdAttribute:"data-test"

We write a test script to navigate back to the sauce demo website, locate the username form field by its data-test attribute and fill in the username placeholder with **rux0422**

